



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Materia: Organización Y Arquitectura de Computadoras

Profesor: M.I. José Antonio de Jesús Arredondo Garza

Práctica 1

Raíz Cuadrada

No. de Cuenta	**Nombre**
320068234	Ayala Hernández María Fernanda
320335163	Salazar Islas Luis Daniel
320078282	Tepal Briseño Hansel Yael
320223822	Ugartechea González Luis Antonio

Fecha de entrega: 5 de octubre del 2026

Calificación: _____

1 Objetivo

En esta práctica se implementó una función para calcular la raíz entera de un número mediante la resta sucesiva de números impares. La idea principal es que la suma de los primeros n números impares es n^2, por lo que al restar $1, 3, 5, 7, \ldots$ se puede contar cuántas restas fueron necesarias para llegar a cero o a un valor menor.

2 Desarrollo

El desarrollo dela práctica consistió en implementar la función de raíz entera en dos lenguajes: MicroPython y ensamblador ARM. La función toma un número entero positivo y devuelve su raíz entera.

El algoritmo funciona de la siguiente manera:

- Inicializa el resto con el valor original.
- Resta secuencialmente $1, 3, 5, 7, \ldots$
- Cada resta exitosa representa una unidad de la raíz.
- Cuando el resto ya no alcanza al siguiente número impar, se termina y se devuelve el contador.

Por ejemplo, para $x = 25$:

$$25 - 1 - 3 - 5 - 7 - 9 = 0$$

Se realizaron 5 restas, por lo que $\lfloor \sqrt{25} \rfloor = 5$.

3 Implementación en Python

La implementación directa en MicroPython permite probar la lógica de manera sencilla. La función `py_odd_sqrt` recibe un número entero y devuelve su raíz entera.

```
1  def py_odd_sqrt(x):
2      remainder = x
3      odd_number = 1
4      root = 0
5
6      while remainder >= odd_number:
7          remainder -= odd_number
8          odd_number += 2
9          root += 1
10
11     return root
```

Listing 1: Implementación principal en Python

4 Implementación en ARM

La función ARM realiza el mismo procedimiento en lenguaje ensamblador, usando registros para llevar el valor restante, el número impar actual y el contador de la raíz. La lógica es equivalente a la versión en Python, pero optimizada para ejecutarse directamente en la placa con la función `odd_sqrt`.

```
@micropython.asm_thumb
def odd_sqrt(r0):
    mov(r4, r0) # Remaining value
    mov(r1, 1) # Current odd number
    mov(r2, 0) # Integer square root

    label(LOOP)
    cmp(r1, r4) # Check if the current odd number is greater than
       the remaining value
    bhi(DONE)

    sub(r4, r4, r1) # Subtract the current odd number from the
       remaining value
    add(r1, r1, 2) # Increment the current odd number by 2
    add(r2, r2, 1) # Counts one successful subtraction, which
       corresponds to the integer square root
    b(LOOP)

    label(DONE)
    mov(r0, r2)
```

Listing 2: Implementación principal en ARM

5 Pruebas y Validación

Esta sección de código ejecuta una serie de pruebas en donde se compara la salida de el calculo obtenido para la solución en Python y la solución en ARM con una serie de valores esperados. Se imprimen los resultados de cada prueba, indicando si la prueba pasó o falló.

```
tests = [
    (0, 0),
    (1, 1),
    (2, 1),
    (3, 1),
```

```
    (4, 2),
    (5, 2),
    (8, 2),
    (9, 3),
    (10, 3),
    (24, 4),
    (25, 5),
    (35, 5),
    (36, 6),
    (50, 7),
    (99, 9),
    (100, 10),
    (1000, 31),
]

print("=" * 42)
print("  INTEGER SQUARE ROOT - ODD NUMBERS")
print("=" * 42)
print("\n[ARM Assembly]")

for x, expected in tests:
    result = odd_sqrt(x)
    if result == expected:
        print("  PASS | input: {:>6} | sqrt: {:>4}".format(x, result
            ))
    else:
        print("  FAIL | input: {:>6} | expected: {:>4} | got: {:>4}"
            .format(x, expected, result))

print("\n[MicroPython]")

for x, expected in tests:
    result = py_odd_sqrt(x)
    if result == expected:
        print("  PASS | input: {:>6} | sqrt: {:>4}".format(x, result
            ))
    else:
        print("  FAIL | input: {:>6} | expected: {:>4} | got: {:>4}"
            .format(x, expected, result))

print("\n" + "=" * 42)
print("                  DONE")
print("=" * 42)
```

Listing 3: Validación de casos de prueba en Python

6 Ejecución en la Raspberry Pi Pico

El archivo principal se puede cargar a la placa con el siguiente comando desde la carpeta raíz del proyecto:

```
./upload.sh practica_01/source/practica_01.py
```

El contenido del script que realiza esta ejecución es el siguiente:

```
#!/usr/bin/env bash

set -euo pipefail

script="${1:?Usage: ./upload.sh SCRIPT.py [DEVICE]}"
device="${2:-/dev/ttyACM0}"

if [[ ! -f "$script" ]]; then
  echo "File not found: $script" >&2
  exit 1
fi

if fuser "$device" >/dev/null 2>&1; then
  echo "Port $device is busy. Stopping the process using it..." >&2
  fuser -k "$device" >/dev/null 2>&1 || true
fi

mpremote connect "$device" fs cp "$script" :main.py
mpremote connect "$device" run "$script"
```

Listing 4: Script de carga a la Pico

Este script:

- valida que el archivo exista,
- libera el puerto si está ocupado,
- copia el programa a la Pico como `:main.py`,
- y lo ejecuta inmediatamente con `mpremote`.

7 Resultados

La salida real obtenida al ejecutar el programa se muestra a continuación:

```
cp practica_01/source/practica_01.py :main.py
Up to date: main.py
==========================================
```

```
  INTEGER SQUARE ROOT - ODD NUMBERS
==========================================

[ARM Assembly]
  PASS | input:       0 | sqrt:     0
  PASS | input:       1 | sqrt:     1
  PASS | input:       2 | sqrt:     1
  PASS | input:       3 | sqrt:     1
  PASS | input:       4 | sqrt:     2
  PASS | input:       5 | sqrt:     2
  PASS | input:       8 | sqrt:     2
  PASS | input:       9 | sqrt:     3
  PASS | input:      10 | sqrt:     3
  PASS | input:      24 | sqrt:     4
  PASS | input:      25 | sqrt:     5
  PASS | input:      35 | sqrt:     5
  PASS | input:      36 | sqrt:     6
  PASS | input:      50 | sqrt:     7
  PASS | input:      99 | sqrt:     9
  PASS | input:     100 | sqrt:    10
  PASS | input:    1000 | sqrt:    31

[MicroPython]
  PASS | input:       0 | sqrt:     0
  PASS | input:       1 | sqrt:     1
  PASS | input:       2 | sqrt:     1
  PASS | input:       3 | sqrt:     1
  PASS | input:       4 | sqrt:     2
  PASS | input:       5 | sqrt:     2
  PASS | input:       8 | sqrt:     2
  PASS | input:       9 | sqrt:     3
  PASS | input:      10 | sqrt:     3
  PASS | input:      24 | sqrt:     4
  PASS | input:      25 | sqrt:     5
  PASS | input:      35 | sqrt:     5
  PASS | input:      36 | sqrt:     6
  PASS | input:      50 | sqrt:     7
  PASS | input:      99 | sqrt:     9
  PASS | input:     100 | sqrt:    10
  PASS | input:    1000 | sqrt:    31

==========================================
                  DONE
==========================================
```

Listing 5: Salida generada por la práctica

8 Conclusiones individuales

8.1 Ayala Hernández María Fernanda

Esta práctica me permitió comprender cómo una Raspberry Pi Pico 2 W puede ejecutar una solución matemática utilizando recursos reducidos. La implementación mostró que estos sistemas pequeños son capaces de realizar tareas concretas de manera eficiente, y que MicroPython facilita cargar, probar y validar un programa directamente en el hardware.

8.2 Salazar Islas Luis Daniel

La implementación en la Raspberry Pi Pico 2 W mostró la importancia de combinar un lenguaje de alto nivel con instrucciones ARM para aprovechar mejor un sistema embebido. Ejecutar el algoritmo directamente en la placa permitió observar que un dispositivo compacto puede procesar datos y responder de forma rápida sin depender de una computadora de propósito general.

8.3 Tepal Briseño Hansel Yael

Con esta práctica comprendí mejor cómo los registros, contadores y saltos condicionales de ARM se utilizan dentro de la Raspberry Pi Pico 2 W. Trabajar con una placa pequeña ayudó a reconocer que la eficiencia del código es importante cuando los recursos son limitados, y que los sistemas embebidos son fundamentales para desarrollar soluciones compactas y de bajo consumo.

8.4 Ugartechea González Luis Antonio

La carga y ejecución del programa en la Raspberry Pi Pico 2 W permitió relacionar directamente el software con el hardware. El uso del script para transferir el archivo y ejecutarlo en la placa demostró que estos sistemas pequeños son prácticos para prototipar, automatizar tareas y construir soluciones especializadas con un costo y consumo menores.